

Atelier 1 : Les collections et la généricité en java

Exercice 1

Partie 1

On souhaite gérer un ensemble d'étudiants. Chaque étudiant sera défini par une classe *Etudiant* et devra présenter les informations suivantes :

- Un matricule ;
- Un nom ;
- Une liste de Notes (de taille indéfinie).

Une classe *Note* permettra de contenir pour chaque cours, l'intitulé du cours ainsi que la note obtenue. Les notes de chaque étudiant seront stockées dans une *ArrayList*.

Une méthode *addNote* permettant d'ajouter une note à l'étudiant sera définie.

Partie 2

On souhaite mettre en place une classe capable de réaliser des statistiques sur une collection d'objets, comme par exemple, des *Etudiants*, des *Notes*, ... Cette classe, qui sera nommée **Stats**, pourra ainsi calculer le maximum, le minimum et la moyenne d'une collection d'objets.

Toutes les classes qui peuvent faire l'objet de statistiques implémenteront une interface **Statisticable**, qui est décrite comme suit :

```
public interface Statisticable {  
    public abstract float getValue() ;  
}
```

Tout objet « statisticable » doit donc avoir une certaine valeur ; pour un *Etudiant*, on choisit de prendre la moyenne de ses notes comme valeur de l'*Etudiant*. La classe *Stats* sera ensuite utilisée et donnera pour :

- Chaque étudiant :
 - sa moyenne ;
 - sa meilleure note ;
 - sa moins bonne note ;
- Chaque groupe d'étudiants :
 - la moyenne du groupe ;
 - le meilleur étudiant ;
- le moins bon étudiant.

Partie 3

a- On souhaite pouvoir classer la liste d'étudiants suivant le matricule. Pour ce faire, on Implémentera l'interface Comparable dans la classe Etudiant. La méthode compareTo devra donc être définie dans la classe Etudiant.

Une fois cela réalisé, on triera la liste d'étudiants à l'aide de la méthode Collections.sort.

a- On souhaite également pouvoir trier la liste d'étudiants par moyenne et par nom.

Dans ce but, deux nouvelles classes (CompareMoyenne et CompareNom) seront créées et implémenteront l'interface Comparator. Ces classes devront donc chacune définir une méthode compare prenant comme arguments les deux objets à comparer et réalisant un traitement similaire à celui de CompareTo dans l'exercice précédent.

Exercice 2 : Table associative : un annuaire

On vous demande d'écrire une classe **Annuaire** pour mémoriser [1]des numéros de téléphone et d'adresses. Chaque entrée est représentée par une fiche à plusieurs champs : un *nom*, un *numéro* et une *adresse*. La structure des fiches est décrite par une classe **Fiche** que vous devez écrire.

Écrivez également une classe **Annuaire** comportant une table associative (**Map<String,Fiche>**) qui sera faite d'associations (*un_nom* , *une_fiche*).

a. Dans un premier temps, la table associative en question sera une instance de la classe **HashMap**.

Écrivez un programme répétant les opérations suivantes

- lecture d'une « commande » d'une des formes : *+nom*, *?nom*, *!* ou *bye*,
- si la commande a la forme *?nom*, recherche et affiche la fiche concernant le *nom* indiqué,
- si la commande est de la forme *+nom*, saisie des autres informations d'une fiche associée à ce nom et insertion de la fiche correspondante dans l'annuaire,
- si la commande est *!*, affichage de toutes les fiches de l'annuaire,
- si la commande est *.* (un point), arrêt du programme.

B. On constate que la commande *!* produit l'affichage des fiches dans un ordre imprévisible. Que faut-il changer dans le programme précédent pour que les fiches apparaissent dans l'ordre des noms ?

Annexe :

Pour utiliser un objet dans une collection triée, on peut :

- Soit utiliser des éléments instances d'une classe comparable,
- Soit spécifier un comparateur d'éléments au constructeur de la collection.

Interface (Comparable)

Une classe implémente l'interface `java.lang.Comparable` en ajoutant la méthode suivante :

L'interface *Comparable*, nécessite l'implémentation de la méthode *compareTo* :

Interface (Comparable)

```
int compareTo(Object o);  
}
```

`int compareTo(Object o)` ou

(Java 5+) `interface Comparable<T>` :

`int compareTo(T o)`

Dans cette méthode l'objet `this` est comparé à l'objet `o`. Cette méthode doit retourner

- -1 si `this < o`
- 0 si `this == o`

Compareur (Comparator)

Une classe implémentant cette interface doit déclarer deux méthodes :

`int compare (Object o1, Object o2)`

`int equals(Object o)`

Ou (Java 5+)

Interface `Comparator<T>` :

`int compare(T o1, T o2)`

`int equals(Object o)`

La méthode `equals` compare les comparateurs (eux-mêmes) `this` et `o`.

La méthode `compare`, compare les deux objets spécifiés et retourne le signe de la soustraction virtuelle `o1 - o2`, comme expliqué dans la section précédente. C'est-à-dire :

- -1 si `o1 < o2`
- 0 si `o1 == o2`
- +1 si `o1 > o2`